PPL LAB 9

Name: Abhinav Singh Bhagtana

Roll no: A-029

Reg no: 230905225

Q1. #include "cuda_runtime.h" #include <stdio.h>

global void csrMultiplyKernel(int *valA, int *colA, int *rowA, int *valB, int *colB, int *rowB, int *C, int m, int p) { int row = blockIdx.x * blockDim.x + threadIdx.x;

```
if (row >= m)
   return;

for (int j = rowA[row]; j < rowA[row+1]; j++)
{
   int k = colA[j];
   int valA_val = valA[j];

   for (int t = rowB[k]; t < rowB[k+1]; t++)
   {
      int colB_idx = colB[t];
      int valB_val = valB[t];

      atomicAdd(&C[row * p + colB_idx], valA_val * valB_val);
   }
}


}
```

void denseToCSR(int *A, int m, int n, int values[], int col_index[], int row_ptr[], int *nnz) { int k = 0; row_ptr[0] = 0;

```
for(int i = 0; i < m; i++)
{
   for(int j = 0; j < n; j++)
   {
```

```
        if(A[i * n + j] != 0)
        {
            values[k] = A[i * n + j];
            col_index[k] = j;
            k++;
        }
    }
    row_ptr[i+1] = k;
}
*nnz = k;


}

int main() { int m,n,k; printf("enter dimensions (m,n,k):\n");
scanf("%d%d%d",&m,&n,&k);

int A[m][n], B[n][k], C[m][k];

int *d_C, *d_Data_A, *d_Data_B, *d_col_idx_A, *d_col_idx_B,
*d_row_ptr_A, *d_row_ptr_B;

printf("enter sparse matrix A:\n");
for(int i = 0;i < m;i++)
    for(int j = 0;j<n;j++)
        scanf("%d",&A[i][j]);

printf("enter sparse matrix B:\n");
for(int i = 0;i < n;i++)
    for(int j = 0;j<k;j++)
        scanf("%d",&B[i][j]);

int Data_A[100]={0}, col_idx_A[100]={0}, row_ptr_A[101]={0}, sizeA;
int Data_B[100]={0}, col_idx_B[100]={0}, row_ptr_B[101]={0}, sizeB;

denseToCSR((int*)A,m,n,Data_A,col_idx_A,row_ptr_A,&sizeA);
denseToCSR((int *)B,n,k,Data_B,col_idx_B,row_ptr_B,&sizeB);
```

```
// CUDA malloc
cudaMalloc(&d_C,sizeof(int)*m*k);
cudaMalloc(&d_Data_A,sizeof(int)*sizeA);
cudaMalloc(&d_Data_B,sizeof(int)*sizeB);
cudaMalloc(&d_col_idx_A,sizeof(int)*sizeA);
cudaMalloc(&d_col_idx_B,sizeof(int)*sizeB);
cudaMalloc(&d_row_ptr_A,sizeof(int)*(m+1));
cudaMalloc(&d_row_ptr_B,sizeof(int)*(n+1));

// copy to device
cudaMemcpy(d_Data_A,Data_A,sizeof(int)*sizeA,cudaMemcpyHostToDevice
);
cudaMemcpy(d_Data_B,Data_B,sizeof(int)*sizeB,cudaMemcpyHostToDevice)
;
cudaMemcpy(d_col_idx_A,col_idx_A,sizeof(int)*sizeA,cudaMemcpyHostToD
evice);
cudaMemcpy(d_col_idx_B,col_idx_B,sizeof(int)*sizeB,cudaMemcpyHostToD
evice);
cudaMemcpy(d_row_ptr_A,row_ptr_A,sizeof(int)*(m+1),cudaMemcpyHostTo
Device);
cudaMemcpy(d_row_ptr_B,row_ptr_B,sizeof(int)*(n+1),cudaMemcpyHostToD
evice);

cudaMemset(d_C,0,sizeof(int)*m*k);

dim3 block(256);
dim3 grid((m + 255)/256);

csrMultiplyKernel<<<grid, block>>>(
    d_Data_A, d_col_idx_A, d_row_ptr_A,
    d_Data_B, d_col_idx_B, d_row_ptr_B,
    d_C,
    m, k);

// copy back
cudaMemcpy(C,d_C,sizeof(int)*m*k,cudaMemcpyDeviceToHost);

// print result
```

```
printf("Result matrix:\n");
for(int i=0;i<m;i++)
{
   for(int j=0;j<k;j++)
      printf("%d ",C[i][j]);
   printf("\n");
}

// free
cudaFree(d_C);
cudaFree(d_Data_A);
cudaFree(d_Data_B);
cudaFree(d_col_idx_A);
cudaFree(d_col_idx_B);
cudaFree(d_row_ptr_A);
cudaFree(d_row_ptr_B);

return 0;


}
```

Output:

```
STUDENT@MIT-ICT-LAB5-29:~/230905225/week9$ ./a.out
enter dimensions (m,n,k):
3 3 3
enter sparse matrix A:
1 0 2
0 3 0
4 0 5
enter sparse matrix B:
7 0 0
0 8 0
9 0 6
Result matrix:
25 0 12
0 24 0
73 0 30
STUDENT@MIT-ICT-LAB5-29:~/230905225/week9$
```

Q2.

#include "cuda_runtime.h" #include <stdio.h> #include <math.h>

global void kernel(int *A, int B, int m, int n) { int row = blockIdx.x * blockDim.x + threadIdx.x; int col = blockIdx.y * blockDim.y + threadIdx.y; if(row >= m || col >= n) return; B[rown + col] = ceil(pow(A[row*n+col],row+1)); }

int main() { int m,n; printf("enter dimensions: "); scanf("%d %d",&m,&n);

```
int A[m][n],B[m][n],*d_A,*d_B;
printf("enter A (%dx%d):\n",m,n);
for(int i = 0;i < m;i++)
   for(int j = 0;j < n;j++)
      scanf("%d",&A[i][j]);

cudaMalloc(&d_A,sizeof(int) * m * n);
cudaMalloc(&d_B,sizeof(int) * m * n);

cudaMemcpy(d_A,A,sizeof(int) * m * n, cudaMemcpyHostToDevice);
dim3 block(16,16);
dim3 grid((m+15)/16.0,(n+15)/16.0);
kernel<<<grid,block>>>(d_A,d_B,m,n);

cudaMemcpy(B,d_B,sizeof(int) * m * n, cudaMemcpyDeviceToHost);

printf("Resultant matrix:\n");
for(int i = 0;i < m;i++){
   for(int j = 0;j < n;j++)
      printf("%d\t",B[i][j]);
   printf("\n");
}
cudaFree(d_A);
cudaFree(d_B);
return 0;


}
```

Output:

```
STUDENT@MIT-ICT-LAB5-29:~/230905225/week9$ nvcc q2.cu
STUDENT@MIT-ICT-LAB5-29:~/230905225/week9$ ./a.out
enter dimensions: 2 3
enter A (2x3):
1 2
3 4
5 6
Resultant matrix:
1       2       3
16      25      36
STUDENT@MIT-ICT-LAB5-29:~/230905225/week9$
```

Q3.

#include "cuda_runtime.h" #include <stdio.h>

global void kernel(int *A, int *B, int m, int n) { int col = blockIdx.x * blockDim.x + threadIdx.x; int row = blockIdx.y * blockDim.y + threadIdx.y; if(row >= m || col >= n) return; if(row != 0 && row != m-1 && col != 0 && col != n-1) { int x = A[row * n + col]; int bits = (x == 0) ? 1 : (32 - __clz(x)); int ones_comp = (~x) & ((1 << bits) - 1); B[row * n + col] = ones_comp; } else B[row * n + col] = A[row * n + col]; } int main() { int m,n; printf("enter dimensions:\n"); scanf("%d%d",&m,&n);

```
int A[m][n],B[m][n],*d_A,*d_B;
printf("enter matrix (%dx%d):\n",m,n);
for(int i = 0;i < m;i++)
    for(int j =0;j<n;j++)
        scanf("%d",&A[i][j]);
int size = sizeof(int)*m*n;

cudaMalloc(&d_A,size);
cudaMalloc(&d_B,size);

cudaMemcpy(d_A,A,size,cudaMemcpyHostToDevice);

dim3 block(16,16);
dim3 grid((m+15)/16.0,(n+15)/16.0);
kernel<<<grid,block>>>(d_A,d_B,m,n);

cudaMemcpy(B,d_B,sizeof(int) * m * n, cudaMemcpyDeviceToHost);

printf("Resultant matrix:\n");
for(int i = 0;i < m;i++){
    for(int j = 0;j < n;j++)
        printf("%d\t",B[i][j]);
    printf("\n");
```

```
}
cudaFree(d_A);
cudaFree(d_B);
return 0;


}
```

Output: